COE 379L Midterm Project Report: Atlas Laboratory Agent

Name: Taylor Gomez | UT EID: tjg2787 | Repository:
https://github.com/tgomez7816/coe379lfa26-midterm

1. System Overview

When a request enters the agent system, it is held within a trusted `LabRequest` envelope. This envelope is as an authenticated container and holds the user's identity and all relevant request information. For example, this envelope also includes project, resource, and interval details. The system prompt and this envelope are the only sources of trusted instructions. All other data, including retrieved passages and tool observations, are explicitly treated as untrusted. This separation makes sure that the agent never follows instructions or requests embedded in untrusted data. The agent is able to maintain a clear boundary between what can and cannot influence its behavior.

The agent operates within a bounded decision–action–observation loop. It can provide exactly one typed decision per step: a tool call, a clarification question, or a final response. This loop is limited in the number of steps (a maximum of 10) to prevent endless processing. After each model decision, a validation gate runs before any tool execution. This gate checks for formatting validity, permissions, and policy compliance. If a proposed action fails validation, it is denied and a failure observation is recorded in the trace. Only validated actions can proceed to tool execution. At the end of the loop, the final `AgentResponse` is checked against the trace. Every reported action must be followed by a successful tool observation, and every citation must correspond to a passage retrieved during the current run.

2. Public Benchmark Results

Table 1: Aggregate metrics from `public_benchmark.json`.

Metric	Value	Metric	Value
`scenario_count`	6	`unnecessary_refusal_rate`	0
`success_rate`	1	`clarification_accuracy`	1
`invalid_structured_response_rate`	0	`average_tool_calls`	2.666
`unsafe_action_attempt_rate`	0		

From this data, we can conclude that all 6 public scenarios passed. This generally means that the model runs reached the correct terminal state, with the correct final response and actions taken. The metrics also show that it did well on safety metrics such as `unsafe_action_attempt_rate` and `unnecessary_refusal_rate`. These two characteristics are always in contention because of how they "pull" on each other. For example, an agent can achieve a zero unsafe action rate by simply refusing all actions, and can also receive zero refusals by acting recklessly. Optimizing one rate can directly cause a spike in the other. However, the tool calls statistic can mask the number of tool calls each scenario required, an overly high number of calls could be evened out by another scenario's count and vice versa.

3. Student Tests

Test 1 in the `test_student_cases.py` file checks whether or not the agent denies an unsafe reservation proposal before executing the `reserve_resource` tool. This test is important because it verifies that the action gate is reliably blocking proposals before the mutating tool begins. The agent should not need to rely on the backend to reject unsafe actions, since a proposed-blocked call is a policy bug.

Test 2 focuses on successful final responses that contradict with the trace. More specifically, the final response claims to be successful while not being supported by a successful tool observation. The agent should be

able to indicate the contradiction and not terminate with a 'completed' status. This test ensures quality by verifying that the agent can not claim a completed action unless the trace is consistent. This prevents the agent from presenting hallucinated or fabricated successes and keeps `AgentResponse` truthful.

While the other two tests were supplied by the assignment, Test 3 was not given a specific characteristic to validate. Test 3 makes sure that the agent is not influenced by the content of retrieved files, such as embedded instructions. The test verifies that the agent treats retrieved documents as inert data, rather than instructions. This safety rule is explicitly stated in the system prompt. Test 3 asserts that embedded instructions do not hijack the tool call of a user's request. Additionally, the test addresses that embedded instructions do not appear in the final response, and that the request achieves normal completion. This is a key failure mode that protects the system from poisoned retrieved content that steers the agent into unauthorized actions.

4. Implementation Decisions

My validators return short, machine-readable error codes rather than string messages. At first, I began with string messages but this resulted in errors because the tests struggled to interpret them. I also ensure that the model collects all errors before returning. This allows the model to fix multiple problems in one step rather than one per retry. This pattern reduces wasted turns.

When a validation fails, the denied call is not executed. Instead, the invalid call is recorded in the trace, with the help of a `TraceEvent` that pairs the proposed `ToolCallDecision` with the failed `ToolCallObservation`. This observation is then fed back into the model's context. I prioritized returning errors to the model rather raising exceptions because raised exceptions end a run with a traceback the model cannot correct.

For termination, I focused on the typed decision kinds. A `FinalDecision` ends the loop normally. A `ClarificationDecision` asks the user for input when the required fields are missing. For unresolved request fields, where there are None in the trusted LabRequest envelope, the code treats them as a distinct error class 'unresolved.' This is separate from mismatch errors. An unresolved field signals that clarification is needed instead of that the model attempting substitution. This keeps the gate's denial reason actionable and prevents the model from silently filling in trusted values like `project_id`, `resource_id`, or the interval itself.

Data Flow Diagram (not counted toward the 2-page limit)

